CHRONOGUARD · OPERATOR'S GUIDE

Dashboard: what every box and feature does

A practical, screen-by-screen explanation of the dashboard, its metrics, charts, filters, detail panels, and test controls.

Prepared from the active dashboard and FastAPI implementation
26 October 2026 · Local application guide

How to use this guide

This guide follows the live dashboard in `index.html` and cross-checks its displayed values and actions against the FastAPI routes. It describes what the current code does, including important limits, rather than assuming every label represents a production capability.

> **Important distinction:** the Simulation Lab deliberately exercises local validators and simulated service dependencies. It does not send external traffic. Server load is simulated workload pressure, not machine CPU utilization.

Contents

1. The shared application frame and refresh behavior
2. System Overview: status cards, charts, monitoring, and recent activity
3. Server Monitoring: health, response time, and check history
4. Incidents & Alerts: filters, incident rows, and details
5. Request Guardian: request outcomes, search, table, and request details
6. Audit Logs: event filters, records, details, and CSV export
7. Simulation Lab: Health Shield, scenarios, live progress, and history
8. Status meanings, data boundaries, and practical operating sequence

Quick orientation

Area	Purpose	Typical question answered
Overview	At-a-glance service and request state	Is the system healthy, and what recently happened?
Servers	Primary/backup health and check evidence	Which simulated dependency is active, and how have checks behaved?
Incidents	Detected problem records and recovery summaries	What failure was recorded and what status/recovery result is stored?
Requests	Validation and processing records	Was a request accepted, corrected, rejected, or otherwise processed?
Audit Logs	Chronological record of request and incident events	What was recorded around a particular request or incident?
Simulation Lab	Controlled backend tests and persisted runs	Does a bounded scenario behave as expected?

The navigation names and active screen heading are generated by the dashboard. The browser page title changes to match the selected screen.

1. Shared application frame

Left navigation and brand

The CG brand identifies ChronoGuard. The navigation switches between Overview, Servers, Incidents, Requests, Audit Logs, and Simulation Lab. The selected screen is highlighted. On narrow screens, the hamburger button opens the side drawer; choosing a page closes it.

Top header controls

Control / indicator	What it means or does
API status	**Checking** is the initial state. **Connected** means the health endpoint responded; **Degraded** indicates a reported database/service health problem; **Disconnected** means the health request failed. **Demo** means the interface is using its local sample data instead of the backend.
Monitoring status	On/Paused is derived from backend monitoring settings. A dash means that status could not be read. This is separate from API connectivity.
Updated time	Shows the browser's local time when the selected screen finished refreshing. It is not a timestamp for every row or an indication that every endpoint succeeded.
Refresh	Runs the current screen's data-loading path again. An in-progress refresh is ignored to avoid overlapping refresh cycles.
Light/Dark mode	Switches the color theme. The preference is saved in browser local storage on that device.
Demo data	Switches dashboard requests to built-in sample records and marks the page as DEMO. It does not read PostgreSQL. Simulation execution is unavailable while demo mode is on.

Banners, loading, errors, and detail drawers

- A purple demo banner warns that visible sample records are not backend/database data.
- A yellow paused-monitoring banner warns that displayed health may be stale; check timestamps.
- Loading messages appear during fetches. If the backend is unreachable, the page explains that no live data is shown and offers Retry.
- Transient toast messages report the result of operations such as manual checks, Shield changes, and simulation actions.
- Clicking a request, incident, or audit row opens a right-side detail drawer. Close with X, Escape, or by clicking the dark backdrop. Focus returns to the element that opened the drawer.

> **Refresh cadence:** outside the Simulation Lab, the app automatically refreshes about every 20 seconds when the tab is visible, no refresh is in progress, no input/select is focused, and the details drawer is closed. Simulation polling is separate and runs about every 350 ms while a run is active.

The default API base is `http://127.0.0.1:8000` (overridable by `CHRONOGUARD_API_BASE`). Dashboard fetches time out after six seconds. Timestamps are displayed in browser-local time; timestamp hover text includes UTC.

2. System Overview

The Overview combines data from the server, overview, request-statistics, incident, log, and request-list endpoints. Some panels can still render if an individual request fails; unavailable data is shown as a dash or empty-state message.

Top KPI cards

Card	What it counts/shows	How to interpret it
Primary server health / Backup server health	Backend-reported health badge and last-check time/failure count.	Health is the monitor's reported state; absent server data is Not Available.
Requests processed	The requests/stats endpoint's total for the Operational scope. Small text shows successful and rejected counts.	This is a record count, not a throughput rate.
Active incidents	Incident records whose status is not marked resolved by the Overview filter. Small text counts critical and warning severities among these.	It reflects recorded incident rows, not an independent live alert detector.
Failover events	Backend count of request records that used the backup server. The secondary value is the current active server when it is backup.	This is a request-based proxy for backup usage, not necessarily a count of distinct switch transitions.
Verification success	Backend percentage of request records whose final status is processed or recovered.	It is unavailable when there are no request records. It is not a separate synthetic verification probe.

System health score

The semicircle gauge displays `health_score` from the overview endpoint, clamped to 0–100. The backend calculates it as the percentage of stored service-health rows currently in a healthy/ok/up/ready state. Green indicates 80–100, amber 60–79, and red below 60; labels are Excellent (90+), Good (80–89), Degraded (60–79), or Critical (<60). If the API supplies no score, the gauge explains that the backend did not compute one.

Failover topology

The diagram shows Client → ChronoGuard → Server A (primary) and Server B (backup) → Database. Server node border colors use the health reported by the backend. A solid route indicates the server is confirmed active; a dashed route indicates standby or unconfirmed. It is a status visualization, not a packet-flow monitor.

Requests over time and Request outcomes

- **Requests over time:** a stacked column chart built from up to 200 operational request rows returned to the page. It groups timestamps into 12 buckets and colors success, corrected, rejected, and other statuses separately. Hover a segment for time/status/count. No timestamped rows produces an explicit empty state.
- **Request outcomes:** a donut chart based on request status statistics. It compares success, corrected, rejected, pending, and failed outcomes. Other/unmapped states are not guaranteed to have their own named slice.

Monitoring and server cards

The Monitoring box displays whether automated checks are enabled/paused and the latest check time/type. Each server card repeats health and provides role, active/standby state, simulated load, readiness, last check, consecutive failures, and response time. Paused monitoring adds a stale-data note.

Self-healing lifecycle

The six stage boxes are Monitor, Detect, Contain, Recover, Verify, and Log. Their descriptions explain the intended lifecycle. A stage badge is only populated when the overview API provides a matching lifecycle value. The current backend overview explicitly provides the monitor state; other stages may show Not Available rather than inferred progress.

Outcome counts and Recent activity

Outcome counts repeats operational status counts and links to Request Guardian. **Recent activity** lists up to six operational log records, newest first, with timestamp, category, message, and outcome; its button opens Audit Logs.

3. Server Monitoring

Server A and Server B cards

Each card represents the backend's `primary` or `backup` service. These are simulated service dependencies in the local monitor implementation.

Field	Meaning
Health badge	Health status from the most recent monitor result; can include healthy, degraded, unhealthy, overloaded, or unknown.
Current role	Active means the persisted active-server setting points here; Standby means it points elsewhere.
Simulated load	Persisted simulated load percentage. It is workload pressure used by controlled scenarios, not host CPU/memory.
Readiness	The backend's readiness/status value, not a separate front-end calculation.
Last health check	Most recent recorded check time, rendered in local time.
Consecutive failures / Response time	Shown when present on the server object. The base `/api/servers` response may not include these values, so the card can show dashes.

Monitor configuration

This box shows the configured check interval, probe timeout, and consecutive-failure threshold. In this code, the monitor defaults are a five-second interval, two-second timeout, and three failed checks before unhealthy; saved configuration can change the threshold. Actual detection duration varies with scheduling and probe timeouts, so the dashboard correctly avoids promising a fixed detection time.

Run manual health check calls the backend's manual check operation immediately and records results. It does not require scheduled monitoring to be enabled. The button is disabled if monitor configuration could not be loaded.

Open Simulation Lab navigates to controlled failure and recovery scenarios.

Response time chart

The chart plots response time from recent health-check history for both services. It is based on stored probe measurements, not client request latency. A missing series/history is shown as an empty state.

Health-check history table

Shows the newest combined checks (up to 50) across both servers.

Column	Explanation
Timestamp	When the health probe was recorded.
Server	Human-readable Server A or Server B name.
Result	Probe result/mode, such as healthy, timeout, or unavailable.
Response	Probe duration in milliseconds, if supplied.

Failures	Consecutive failure count at that check.
Error	Recorded probe error, if any.
State after	Health state after applying the failure threshold.
Type	Scheduled or manual check.

> **Staleness:** if monitoring is paused, stored values remain visible. Use the last-check time and paused warning; do not treat old health as a current probe.

4. Incidents & Alerts

Summary cards

- **Active:** incidents whose status text does not contain “resolved”.
- **Critical / Warnings:** active incidents whose severity text matches critical / warning.
- **Resolved:** incidents whose status contains “resolved”.
- **Failed recoveries:** incidents whose recovery-result text contains “fail”.

These counts are recalculated from the incidents returned by the API. Severity/status comparisons use the returned text; they are not separate alert rules.

Search and filters

Search filters incident ID or title as you type. Severity can be All, critical, warning, or info. Status can be All, open, or resolved. Clear filters resets the text and both selectors. Filtering happens in the browser on the loaded incident list.

Incident table

Column	What it displays
ID / Detected	Incident identifier and its creation/detection timestamp.
Title / Server	Incident type/title and affected service.
Severity / Source	Severity badge and originating component when supplied.
Status / Recovery	Stored incident state and recovery result/action text.
Origin	Operational or Simulation label when available; simulation rows receive a purple marker.

Select a row or press Enter to open the incident drawer.

Incident detail drawer

The drawer includes a six-step lifecycle strip (Detected, Investigating, Contained, Recovering, Verified, Resolved), incident summary, affected service, severity, source/origin, failure details, containment, recovery attempts/result, verification, and up to ten related audit messages.

> **Lifecycle limitation:** the strip selects a position by matching the incident's status text to a stage name. It does not reconstruct a full timeline from events. If the stored status is simply “open”, the strip may not identify a stage. Some summary fields can be blank because the current incident API does not return every field shown by the drawer.

An empty incident page means no incident records were returned; the UI does not synthesize incidents simply because a request was rejected.

5. Request Guardian

Request KPI cards

Card	Meaning
Total requests	Request records counted by the statistics endpoint for the selected scope.
Successfully processed	Records reported under the backend's success status.
Corrected / Rejected / Pending	Counts in each corresponding final-status bucket.
Processing failures	Count with failed status; it is separate from a controlled validation rejection.

Database Monitor, scope, rates, and filters

“Live view” describes the records fetched from the backend, not a streaming socket. Correction and rejection rates are the corresponding status count divided by the total in the chosen scope, displayed to one decimal place. Scope choices are Operational (default), Including simulation, and Simulation only. Search matches request ID or detected issue. Date fields filter received date. Clear filters restores the default page, status, search, scope, sort, and dates.

Status chips select All, Successful, Corrected, Rejected, Pending, Failed, or Quarantined. The selected chip is highlighted. The API request is reloaded when a filter changes.

Request table

Column	What it means
Request ID / Received	Identifier, simulation marker if applicable, and received time.
Original data	Shortened JSON payload. Sensitive keys matching password/secret/token/API-key/credential patterns are redacted before display.
Validation / Detected issue	Validation status and the reported validation/problem text.
Correction applied / Original value / Corrected value	Whether a correction was applied and the before/after value when the API provides it.
Final status	Normalized badge for success, corrected, rejected, failed, pending, quarantined, or unknown state.
Server / Verification / Last updated	Processing destination, verification status, and most recent update timestamp when present.
Actions	View opens the same detail panel as selecting the row.

Request ID, Received, Final status, and Last updated headers are sortable. Click once for descending order; clicking the current column again toggles ascending/descending. Results show returned count, matching count, total count, and 10-row paging controls.

Request detail drawer

Sections show request information, redacted payload, validation results/errors, correction history (type, values, reason, verification), rejection information (reason, failed rule, retry eligibility), and stored audit events. The

related-audit button opens Audit Logs filtered to that request. Missing API fields display as dashes; “No history events stored” means the history endpoint returned no rows, not necessarily that no event occurred outside the stored log.

> A rejection is usually a validation/policy outcome; a processing failure means execution failed. Read both the final-status badge and the issue/reason before deciding what happened.

6. Audit Logs

Event KPI cards

Total recorded events is the backend's total number of stored request and incident rows, not just the current filtered page. The health-check, validation, incident, and recovery cards use category counts supplied with the filtered results; a missing category displays zero when counts are present.

Search and filter controls

Control	Behavior
Search	Searches event text for message, request ID, or incident ID; the backend searches the serialized event fields.
Category	All categories or health_check, validation, correction, rejection, incident, recovery, monitor, simulation.
Severity	All, info, warning, or critical.
Scope	Operational, Including simulation, or Simulation only.
From / To	Inclusive calendar-date bounds on event timestamp.
Clear filters	Restores the initial operational scope and clears all filters/page state.
Export CSV	Downloads a CSV using the current filters; formula-like values are prefixed to reduce spreadsheet formula execution, and quotes are escaped.

Audit event table

Ten rows per page, newest records first. Columns are Event ID, Timestamp, Category, Severity, Source, Server, Request/incident ID, Action, Outcome, and Message. Simulation origin gets a marker. Select a row to open all fields for that event; object values are JSON-serialized with sensitive key names redacted.

The related-request shortcut from a request detail panel sets the request ID filter and switches scope to all origins, so both operational and simulation records can be inspected.

> **Current CSV limit mismatch:** the browser asks the logs endpoint for 1,000 rows in one page, while the backend currently validates `page_size` at a maximum of 500. On the current backend this can make Export CSV fail with a validation response. The guide documents observed behavior; it does not change the application.

The audit endpoint currently builds its feed from stored request and incident records. Do not assume every monitor check or simulation action becomes an audit row unless a corresponding stored event is present.

7. Simulation Lab: control center

This page requires the real backend and PostgreSQL-backed settings/history. With Demo data on, the page intentionally shows that real simulation cannot run.

Summary cards

Card	Meaning
Overall simulation	RUNNING if any run is queued/running; otherwise the newest run outcome, or IDLE when none exist.
ChronoGuard Health Shield	Persisted enabled/disabled Shield setting.
Server A / Server B	Current reported health, active/standby role, and simulated load percent.
PostgreSQL	Database connectivity result from the health endpoint.
Total simulations / Passed tests / Failed tests / Running tests	Counts from the returned persisted run history; failed includes FAIL and ERROR, running includes QUEUED and RUNNING.

Health Shield control

The switch changes the saved `shield_enabled` setting. Disabling prompts for confirmation because it also pauses automated monitoring through the backend setting update. It is not the same as starting/stopping an individual test. The box also shows whether automated checks are running/paused, consecutive failure threshold, simulated-load failover threshold, monitor state, and latest check. Values are read from persisted configuration/status.

Run manual diagnostic check runs a check immediately and refreshes the control center. It is available independently of the periodic schedule.

Preloaded scenarios and safety limits

Scenario cards are grouped by category and contain ID, severity, availability, description, expected result, latest actual result, execution status/progress, configuration, and action buttons. Unsupported scenarios remain visible for planning but have no Run button and are never reported as passed.

> **Boundaries:** implemented load scenarios cap work at 500 local operations and concurrency 10. This is bounded in-process work; no external traffic is generated. Scenario control fields are validated as integer ranges before start and validated again by the backend.

Category	Examples of scenario intent	Implementation status in current backend
Request flooding/load	Bounded validation work, gradual load, queue capacity, queue drain (A1-A4).	Implemented.
Request validation	Malformed email/timestamp, missing fields, wrong type, malformed JSON (B1-B5).	Implemented. Duplicate request and oversized payload cases (B6-B7) are planning-only.
Server failure/failover	Primary/backup unavailable, timeouts, intermittent failure, recovery/failback (C1-C6).	Implemented against simulated dependency probes.

Database/data integrity	Concurrent writes and tagged read-back (D4-D5).	D4-D5 implemented. Isolated database outage/write/latency injection (D1-D3) is not implemented.
Health monitoring/recovery	Configuration, threshold, restore/recovery, repeated failures (E1-E3, E5).	Implemented. Recovery-failure injection (E4) is not implemented.
Security/API robustness	Harmless SQL-like input (F1).	F1 implemented. Authorization, browser CORS, and ASGI method tests (F2-F4) are not implemented here.
End-to-end self-healing	Workflow, incident lifecycle, dashboard consistency, reset behavior (G1-G4).	Implemented as local bounded scenarios.

7. Simulation Lab: running and reading a test

Scenario configuration

Where a supported scenario exposes controls, numeric fields show allowed minimum/maximum values. The current controls include Requests (default 100, 1–500), Concurrency (default 2, 1–10), Queue capacity (default 25, 1–500), and Failure threshold (saved/default threshold, 1–10). Only controls declared by that scenario are shown. Browser validation rejects blank, non-integer, or out-of-range values before submitting.

Run Simulation and Stop

1. Choose a supported scenario and set any available bounds.
2. Select **Run Simulation**. The browser posts scenario ID plus config; a run is persisted and executed by the backend.
3. While a run is queued/running, the run button is disabled and a Stop button is offered. Stop requests cooperative cancellation; it does not undo completed work.
4. The page polls the run record, refreshes its current progress, and continues polling until the status is terminal.

Live simulation progress box

Appears when a run is queued/running. It shows status, scenario ID, start time, elapsed milliseconds, progress bar and percentage, lifecycle stage, and available metrics:

- Requests sent, accepted, rejected, failed, pending, and recovered/retried.
- Average response time and throughput per second.
- Server A and B health/role snapshots.
- Database operation: verified, failed, in progress, or not required for that scenario.

A dash means the run has not supplied that metric; it does not mean zero. Progress and lifecycle stages are backend run metadata, not a prediction of completion time.

Simulation history table

Lists up to the latest 100 persisted runs: shortened run ID, scenario name, start time, status, lifecycle stage, progress, expected result, actual result/error, and outcome. A run's outcome is a comparison/result label; inspect actual result and error before treating a pass as proof beyond that scenario's test.

Refresh data and Reset simulation state

- **Refresh data** reloads Shield settings, monitor status, server state, database health, scenario catalog, and up to 100 run records.
- **Reset simulation state** requires confirmation. The backend deletes simulation run history, restores simulated primary/backup modes to healthy, selects primary as active, zeroes simulated loads, and performs a health check. It explicitly retains request logs and incidents.

> **Reset is not a general database wipe.** It is intended to reset simulation state/history, not remove normal request or incident records. Confirm with the prompt before proceeding.

8. Status meanings and safe reading

Common request outcomes

Status	Dashboard reading
SUCCESS	Request completed successfully according to the final status returned.
CORRECTED	A permitted correction was applied; review original/corrected values and reason.
REJECTED	Validation or policy prevented acceptance; review rejection reason/failed rule.
PENDING	Processing is ongoing or waiting in a recognized intermediate state.
FAILED	Processing failed; distinguish this from a deliberate validation rejection.
QUARANTINED / UNKNOWN	Nonstandard or unrecognized state. Check raw details and backend response.

Common service/simulation signals

- **Healthy/OK/Up/Ready/Verified:** shown in a positive color.
- **Degraded/Warning:** a caution state; investigate latest check, failure count, and error.
- **Unhealthy/Down/Failed/Critical:** a problem state.
- **Simulation PASS/FAIL/ERROR:** result of a particular bounded scenario; not a guarantee of production availability.
- **Unknown/Not Available:** field not returned, not computed, or not recognized. It is not equivalent to healthy.

Recommended operator sequence

1. Check the API status, monitoring status, and whether DEMO is shown.
2. On Overview, compare each server's last-check time with the current time; use the topology and active/standby labels to understand routing state.
3. Open Servers for probe history and failure counters when a health badge is degraded/unhealthy.
4. Open Incidents to inspect the recorded problem and recovery summary.
5. Open the matching request in Request Guardian to examine validation/correction/rejection fields; follow its audit link when needed.
6. Use Simulation Lab only for controlled testing. Review scenario support status and bounded settings before running; do not interpret sample data as measurements.

Data and interpretation boundaries

- The dashboard primarily displays persisted backend records. A missing field is commonly rendered as a dash; an endpoint failure may leave only one panel unavailable.
- Demo records are hard-coded examples with changing relative timestamps. They are not live and cannot be used to validate current system health.
- Health checks target the application's simulated primary/backup modes in the current monitor. They should not be interpreted as checks of arbitrary real infrastructure.
- Charts summarize fetched data; they do not imply continuous telemetry, complete historical retention, or a full request rate.

- “Failover events” is derived from requests processed on backup in the current overview API, and therefore is not necessarily the number of transition events.
- Security-sensitive payload keys are redacted in request details and event-object rendering, but users should still avoid entering secrets into test data.

Appendix: page-to-data map

Screen / feature	Backend data or action	Notes
API indicator	`GET /api/health`	Checks API/database response.
Monitoring indicator and banners	`GET /api/monitor/status`	Monitoring is enabled only when Shield and monitoring settings allow it.
Overview cards/charts	`/api/servers`, `/api/overview`, `/api/requests/stats`, `/api/incidents`, `/api/logs`, `/api/requests`	Some calls fail independently; unavailable pieces can render separately.
Server history/manual check	`/api/servers/{service}/health-history`, `POST /api/monitor/check`	History is persisted probe data.
Incidents	`GET /api/incidents` and incident-filtered logs	Drawer fields depend on returned record attributes.
Requests	`/api/requests/stats`, `/api/requests`, `/api/requests/{id}`, `/api/requests/{id}/history`	Filtered list is paged at ten rows.
Audit Logs	`/api/logs`	CSV is generated in the browser from fetched rows.
Health Shield settings	`GET/PUT /api/shield/config`	Persisted setting; disabling also pauses scheduled monitoring.
Scenario catalog/history	`/api/simulations/scenarios`, `/api/simulations/history`	History and scenarios require backend support.
Run, poll, stop, reset	`POST /api/simulations`, `GET /api/simulations/{run_id}`, `POST .../stop`, `POST /api/simulations/reset`	All run operations target the local backend; reset retains request and incident history.

In one sentence

Overview summarizes → Servers supplies health evidence → Incidents describe recorded failures → Requests explain individual outcomes → Audit Logs connect stored events → Simulation Lab tests bounded behavior.

This document describes the implementation present in the repository when it was generated. If dashboard or API code changes, refresh this guide so field meanings and endpoint behavior remain accurate.